



Day 5

CSS Introduction (Cascading Style Sheets)

1. What is CSS?

CSS is used to design and style HTML web pages.

HTML=Structure

CSS = Design / Look / Styling

Simple Meaning:

CSS makes your website:

- Colorful
- Attractive
- Professional
- Responsive

Example:

Without CSS:

- Plain black text

With CSS:

- Colored text, spacing, layouts, fonts, buttons

2. Why CSS is Important?

Makes website beautiful

Controls layout (position, spacing)

Improves user experience

Used in all real-world websites

Example websites:

- Amazon
- YouTube
- Instagram

All use CSS heavily.

3. Ways to Add CSS in HTML

There are 3 types of CSS:

1. Inline CSS

CSS written inside HTML tag using style attribute.

Syntax:

```
<h1 style="color:red;">Hello World</h1>
```

Example:



```
<p style="color:blue; font-size:20px;">
```

This is inline CSS example

```
</p>
```

Key Points:

- Used inside tag
- Applies only to that element
- Not reusable
- Not recommended for big projects

2. Internal CSS

CSS written inside `<style>` tag in HTML `<head>` section.

Syntax:

```
<head>
```

```
  <style>
```

```
    h1 {
```

```
      color: green;
```

```
    }
```

```
  </style>
```

```
</head>
```

Example:

```
<html>
```

```
<head>
```

```
  <style>
```

```
    p {
```

```
      color: purple;
```

```
      font-size: 18px;
```

```
    }
```

```
  </style>
```

```
</head>
```

```
<body>
```

```
  <p>This is internal CSS</p>
```

```
</body>
```

```
</html>
```

Key Points:

- Used inside same HTML file



- Applies to full page
- Better than inline CSS

3. External CSS (MOST IMPORTANT)

CSS written in a separate .css file.

Example Structure:

HTML File

```
<html>
<head>
  <link rel="stylesheet" href="style.css">
</head>
```

```
<body>
  <h1>Hello CSS</h1>
</body>
</html>
```

CSS File (style.css)

```
h1 {
  color: red;
}
```

Key Points:

- Best practice
- Reusable for multiple pages
- Clean and professional
- Used in real projects

4. CSS Selectors (VERY SIMPLE EXPLANATION)

Selectors are used to **target HTML elements**.

Meaning:

"Which element should CSS style?"

1. Element Selector

Targets tag name directly.

Example:

```
h1 {
  color: blue;
}
```

☞ All <h1> tags will become blue

2. ID Selector



Targets single unique element.

Syntax:

```
#title {  
    color: red;  
}
```

HTML:

```
<h1 id="title">Hello</h1>
```

Key Points:

- Unique
- Only one per page

3. Class Selector

Used for multiple elements.

Syntax:

```
.box {  
    color: green;  
}
```

HTML:

```
<p class="box">Text 1</p>  
<p class="box">Text 2</p>
```

Key Points:

- Reusable
- Used in most projects

10 Coding Tasks (Day 5 Practice)**Task 1**

Apply inline CSS to change text color of heading

Task 2

Create internal CSS to style paragraph text

Task 3

Create external CSS file and link it

Task 4

Style all <h1> elements using element selector

Task 5

Create a class .text and apply it to 3 paragraphs

Task 6

Create ID selector for a title

Task 7

Style a login page heading using external CSS



Task 8

Change background color of page using CSS

Task 9

Create a page using all 3 CSS types

Task 10 (Challenge)

Create a simple webpage:

- Title (styled using ID)
- Paragraphs (class styling)
- Background color (external CSS)
- Button with inline CSS

Mini Projects

Project 1: Simple Styled Profile Page

- Name (styled with CSS)
- Bio section
- Background color

Project 2: Login Page UI

- Username input
- Password input
- Login button
- CSS styling (center alignment optional)

Project 3: Basic Website Home Page

- Header
- Paragraph content
- Navigation links styled using CSS

Project 4: Portfolio Intro Page

- Name heading
- About section
- Skills list styled with class selector

Day 6

CSS Box Model (Very Important for UI Design)

1. What is Box Model?

Every HTML element is treated like a **box** in CSS.

That box has 4 parts:

1. Content (text/image)
2. Padding (space inside)

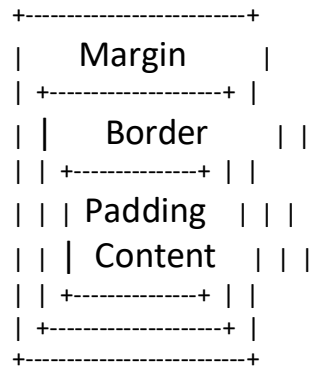


3. Border (outline)
4. Margin (space outside)

Simple Meaning:

Box Model = How space is arranged around an element

2. Box Model Structure



3. Content

This is the actual text or image inside the element.

```
<div>
  Hello World
</div>
```

4. Width & Height

Used to control size of element.

Example:

```
div {
  width: 200px;
  height: 100px;
}
```

5. Border

Border is the outline around the content.

Syntax:

```
div {
  border: 2px solid black;
}
```

Border Types:



border: 2px solid red;
border: 2px dashed blue;
border: 2px dotted green;

6. Padding

Padding is space inside the box (between content and border).

Example:

```
div {  
    padding: 20px;  
}
```

Visualization:

Border

↓

```
+-----+  
| Padding |  
| Content |  
+-----+
```

Padding Types:

```
padding: 10px;           /* all sides */  
padding: 10px 20px;      /* top-bottom | left-right */  
padding: 10px 20px 30px 40px; /* top right bottom left */
```

7. Margin

Margin is space outside the box.

Example:

```
div {  
    margin: 20px;  
}
```

Meaning:

Creates distance between elements

Margin Types:



margin: 10px;
margin: 10px 20px;
margin: 10px 20px 30px 40px;

Box Model Example (Full Code)

```
<html>
<head>
<style>
.box {
  width: 200px;
  height: 100px;
  border: 2px solid black;
  padding: 20px;
  margin: 30px;
  background-color: lightblue;
}
</style>
</head>

<body>

<div class="box">
  Hello Box Model
</div>

</body>
</html>
```

Tasks (Day 6 Practice)

Task 1

Create a box with width 200px and height 200px

Task 2

Add border to a div

Task 3

Add padding of 20px inside a box

Task 4



Add margin between two boxes

Task 5

Create a box with background color and border

Task 6

Create a card with width, padding, border

Task 7

Create 3 boxes with margin between them

Task 8

Use different border styles (solid, dashed, dotted)

Task 9

Create a centered box using margin

Task 10 (Challenge)

Create a profile card with:

- Name
- Bio
- Image placeholder
- Border, padding, margin styling

Mini Projects

Project 1: Profile Card Design

- Name
- Role
- Description
- Box styling with padding & border

Project 2: Image Card Layout

- Image box
- Title below image
- Padding + margin design

Project 3: Product Card

- Product name
- Price
- Description
- Styled box

Project 4: Simple Website Layout Box System



- Header box
- Content box
- Footer box with spacing

Day 7

CSS Layout Basics (Display + Positioning)

1. What is CSS Layout?

CSS Layout decides:

- How elements are arranged on a webpage
- Where elements will appear
- How elements behave in space

Simple Meaning:

Layout = Website structure arrangement

2. Display Property (VERY IMPORTANT)

The display property controls how an element behaves on the page.

2.1 display: block

Block elements always start on a **new line** and take full width.

Example:

```
div {  
  display: block;  
}
```

Characteristics:

- Starts on new line
- Takes full width
- Height/width can be set

Examples of block elements:



<div>
<h1>
<p>
<section>

2.2 display: inline

Inline elements stay in the same line.

Example:

```
span {  
    display: inline;  
}
```

Characteristics:

- No new line
- Takes only required space
- Width/height NOT applied

Examples:

-
- <a>
-

2.3 display: inline-block

Combination of inline + block

Example:

```
div {  
    display: inline-block;  
    width: 100px;  
    height: 100px;  
}
```

Characteristics:

- Same line (like inline)
- Width/height allowed (like block)

2.4 display: none

Completely hides element.

Example:

```
div {  
    display: none;  
}
```

Meaning:

- Element disappears
- No space is taken

Display Comparison Table

Type	Line Break	Width/Height	Space
block	Yes	Yes	Takes full width
inline	No	No	Small space
inline-block	No	Yes	Custom space
none	Hidden	No	No space

3. CSS Positioning (VERY IMPORTANT)

Position property decides **where** an element is placed.

3.1 position: static (default)

```
div {  
    position: static;  
}
```

Meaning:

- Default position
- No movement control (top, left won't work)

3.2 position: relative



Moves element relative to its original position.

```
div {  
  position: relative;  
  top: 20px;  
  left: 30px;  
}
```

Meaning:

- Moves from original position
- Space is still reserved

3.3 position: absolute

Moves element relative to nearest positioned parent.

```
div {  
  position: absolute;  
  top: 50px;  
  left: 100px;  
}
```

Important:

- Takes element out of normal flow
- Overlaps other elements

Parent rule:

If parent has `position: relative`, child absolute works inside it.

3.4 position: fixed

Fixed to screen even when scrolling.

```
div {  
  position: fixed;  
  top: 0;  
}
```

Use cases:

- Navbar



- Chat button
- Floating icons

3.5 position: sticky

Combination of relative + fixed.

```
div {  
  position: sticky;  
  top: 0;  
}
```

Meaning:

Scroll until a point → then sticks

Position Summary Table

Position	Behavior
static	Default
relative	Moves from original
absolute	Moves inside parent
fixed	Stays fixed on screen
sticky	Becomes fixed after scroll

5. Example (All Position Types)

```
<html>  
<head>  
<style>  
  .box1 {  
    position: relative;  
    top: 10px;  
    background: lightblue;  
  }  
  .box2 {  
    position: absolute;  
    top: 50px;  
    left: 100px;  
  }  
</style>  
</head>  
<body>  
  <div class="box1">  
    <div class="box2">  
    </div>  
  </div>  
</body>  
</html>
```



```
        background: pink;
    }
    .box3 {
        position: fixed;
        top: 0;
        right: 0;
        background: yellow;
    }
</style>
</head>
<body>
<div class="box1">Relative Box</div>
<div class="box2">Absolute Box</div>
<div class="box3">Fixed Box</div>
</body>
</html>
```

Tasks (Day 7 Practice)

Task 1

Create 3 block elements and observe line breaks

Task 2

Use inline display for span elements

Task 3

Create inline-block boxes with width and height

Task 4

Hide an element using display:none

Task 5

Create a relative positioned box and move it

Task 6

Create an absolute box inside a parent container

Task 7

Create a fixed navbar at top of page

Task 8

Create sticky header while scrolling

Task 9



Combine display + position in one layout

Task 10 (Challenge)

Create a layout with:

- Header (fixed)
- Sidebar (absolute)
- Content (block layout)
- Footer

Mini Projects

Project 1: Sticky Navbar Website

- Navbar sticks on scroll
- Content below

Project 2: Profile Card Layout

- Inline-block cards
- Spacing using display property

Project 3: Image Gallery Layout

- Multiple images in inline-block format
- Hover spacing (optional next day)

Project 4: Dashboard Layout

- Sidebar (absolute)
- Header (fixed)
- Content area (relative layout)



Day 8

CSS Flexbox Basics (Flex Container & Items)

1. What is Flexbox?

Flexbox (Flexible Box Layout) is a modern CSS layout system used to:

- Align elements easily
- Distribute space
- Control alignment (horizontal + vertical)

Simple Meaning:

Flexbox = Smart layout system to arrange items in a row or column easily

2. Why Flexbox is Important?

Before Flexbox:

- Positioning was difficult
- Float / inline-block was messy
- Centering elements was hard

After Flexbox:

- Easy alignment
- Responsive layouts
- Clean code

Used in:

- Dashboards
- Navbar
- Card layouts
- E-commerce grids

3. Flexbox Two Main Parts

Flexbox has **2 parts**:

1. Flex Container (Parent)
2. Flex Items (Children)

4. Flex Container



The parent element where Flexbox is applied.

Syntax:

```
.container {  
    display: flex;  
}
```

Meaning:

Once display: flex is applied:

- All child elements become flex items
- They arrange in a row by default

Example:

```
<div class="container">  
    <div>Box 1</div>  
    <div>Box 2</div>  
    <div>Box 3</div>  
</div>  
  
.container {  
    display: flex;  
}
```

Default Behavior of Flexbox

Items arrange in **row direction**

Box1 → Box2 → Box3

5. Flex Direction (VERY IMPORTANT)

Controls direction of items.

row (default)

```
.container {  
    display: flex;  
    flex-direction: row;  
}
```

column

```
.container {  
  display: flex;  
  flex-direction: column;  
}
```

row-reverse

flex-direction: row-reverse;

column-reverse

flex-direction: column-reverse;

Flex Direction Summary

Value	Direction
row	Left → Right
column	Top → Bottom
row-reverse	Right → Left
column-reverse	Bottom → Top

6. Flex Items

Child elements inside flex container.

```
<div class="container">  
  <div class="item">1</div>  
  <div class="item">2</div>  
</div>
```

7. Main Flex Properties (Container Level)**justify-content (Horizontal Alignment)**

Controls alignment in main axis (row direction)

Example:

```
.container {  
  display: flex;  
  justify-content: center;  
}
```

Values:

flex-start



Items start from left

flex-end

Items move to right

center

Items center

space-between

Equal space between items

space-around

Equal space around items

Example Visual:

flex-start → [1][2][3]

center → [1][2][3]

space-between → [1] [2] [3]

align-items (Vertical Alignment)

Controls alignment in **cross axis**

Example:

```
.container {  
  display: flex;  
  align-items: center;  
}
```

Values:

stretch (default)

Full height

flex-start

Top alignment

flex-end

Bottom alignment

center

Middle alignment

7. Gap Property

```
Used to give space between items..container {  
    display: flex;  
    gap: 20px;  
}
```

Advantage:

No need for margin between items

8. Full Flexbox Example

```
<html>  
<head>  
<style>  
  .container {  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    gap: 20px;  
    height: 200px;  
    background-color: lightgray;  
  }  
  .box {  
    width: 80px;  
    height: 80px;  
    background-color: steelblue;  
    color: white;  
    display: flex;  
    justify-content: center;  
    align-items: center;  
  }  
</style>  
</head>  
<body>  
  <div class="container">  
    <div class="box">1</div>  
    <div class="box">2</div>  
    <div class="box">3</div>
```



```
</div>  
</body>  
</html>
```

Tasks (Day 8 Practice)

Task 1

Create flex container with 3 boxes

Task 2

Align items in center using flex

Task 3

Change flex-direction to column

Task 4

Use justify-content: space-between

Task 5

Add gap between flex items

Task 6

Create navbar using flexbox

Task 7

Center a login form using flexbox

Task 8

Create card layout using flexbox

Task 9

Build horizontal menu using flexbox

Task 10 (Challenge)

Create a dashboard layout:

- Sidebar (left)
- Content area (right)
- Use flexbox for main layout

Mini Projects

Project 1: Responsive Navbar

- Logo left
- Menu right
- Flexbox alignment

Project 2: Login Page Center Layout

- Center form using flexbox
- Vertical + horizontal alignment

Project 3: Product Card Layout



- 3–4 cards in row
- Gap spacing

Project 4: Dashboard Layout

- Sidebar + main content
- Flex container layout system

Day 9

Flexbox Advanced (Alignment + justify-content Deep Dive)

1. Flexbox Recap (Quick)

Flexbox has 2 axes:

- Main Axis (horizontal by default → row)
- Cross Axis (vertical)

So alignment happens in 2 ways:

- Horizontal alignment → justify-content
- Vertical alignment → align-items

2. justify-content (MAIN AXIS CONTROL)

This controls how items are arranged **horizontally (by default)** inside a flex container.

Basic Syntax:

```
.container {  
  display: flex;  
  justify-content: flex-start;  
}
```

2.1 flex-start (DEFAULT)

justify-content: flex-start;

Items start from left side

[1][2][3]

2.2 flex-end



`justify-content: flex-end;`

Items move to right side

2.3 center

`justify-content: center;`

Items go to center

`[1][2][3]`

2.4 space-between (VERY IMPORTANT)

`justify-content: space-between;`

First item → left

Last item → right

Equal space between items

`[1] [2] [3]`

2.5 space-around

`justify-content: space-around;`

☞ Equal space around each item

`[1] [2] [3]`

2.6 space-evenly (MODERN ★)

`justify-content: space-evenly;`

Equal space everywhere

`[1] [2] [3]`

justify-content Summary Table

Value	Behavior
<code>flex-start</code>	Left align
<code>flex-end</code>	Right align
<code>center</code>	Center align
<code>space-between</code>	Ends fixed, middle spaced
<code>space-around</code>	Equal around items
<code>space-evenly</code>	Perfect equal spacing

3. align-items (CROSS AXIS CONTROL)

Controls vertical alignment (inside container height)

Basic Syntax:



```
.container {  
    display: flex;  
    align-items: center;  
}
```

3.1 stretch (DEFAULT)

align-items: stretch;

Items stretch full height of container

3.2 flex-start

align-items: flex-start;

Items align to top

[1]

[2]

[3]

3.3 flex-end

align-items: flex-end;

Items align to bottom

3.4 center (VERY IMPORTANT)

align-items: center;

Items vertically centered

[1]

[2]

[3]

align-items Summary Table

Value	Behavior
stretch	Full height
flex-start	Top
flex-end	Bottom
center	Middle

4. justify-content + align-items Together (REAL USE)

```
<html>
<head>
<style>
.container {
  display: flex;
  justify-content: center;
  align-items: center;
  height: 300px;
  background: lightgray;
}
.box {
  width: 80px;
  height: 80px;
  background: steelblue;
  color: white;
  display: flex;
  justify-content: center;
  align-items: center;
}
</style>
</head>
<body>
<div class="container">
  <div class="box">A</div>
  <div class="box">B</div>
  <div class="box">C</div>
</div>

</body>
</html>
```

5. Real-Life Understanding

Property	Real Life Meaning
justify-content	Arrange chairs left/right
align-items	Adjust height position of chairs



Tasks (Day 9 Practice)

Task 1

Create flex container with justify-content: center

Task 2

Use flex-end and observe alignment

Task 3

Apply space-between for 3 boxes

Task 4

Use space-around in navigation menu

Task 5

Center items vertically using align-items

Task 6

Create a navbar with space-evenly menu items

Task 7

Build a centered login form using both properties

Task 8

Create card layout with space-between alignment

Task 9

Design header with logo left and menu right using flex

Task 10 (Challenge)

Create a dashboard header:

- Logo (left)
- Search bar (center)
- Profile icon (right)

Use flexbox alignment properly

Mini Projects

Project 1: Modern Navbar

- Logo left
- Menu center
- Button right
- Proper spacing using justify-content

Project 2: Centered Login Page

- Fully centered form (horizontal + vertical)

Project 3: Product Grid Header

- Title left
- Filter options right

Project 4: Dashboard Header Layout



- Logo + search + profile alignment using flexbox

Day 10

CSS Grid Layout Basics (Grid Container, Columns & Rows)

1. What is CSS Grid?

CSS Grid is a **2D layout system** used to design web pages in:

- Rows (horizontal)
- Columns (vertical)

Flexbox = 1D (row OR column)

Grid = 2D (row AND column together)

Simple Meaning:

Grid = Table-like layout system for modern websites

Used in:

- Dashboard layouts
- E-commerce product grids (Amazon style)
- Image galleries
- Admin panels

2. Grid Container

The parent element where grid is applied.

Syntax:

```
.container {  
  display: grid;  
}
```

Once applied:

- Child elements become grid items
- We can control rows & columns

Example HTML:

```
<div class="container">  
  <div>1</div>  
  <div>2</div>  
  <div>3</div>  
  <div>4</div>  
</div>
```

3. Grid Columns

Columns define **how many items in one row**

Syntax:

```
.container {  
    display: grid;  
    grid-template-columns: 100px 100px 100px;  
}
```

Meaning:

- 3 columns
- Each column width = 100px

Output:

```
[1][2][3]  
[4]
```

4. Repeat Function (VERY IMPORTANT)

Instead of writing many values:

Syntax:

```
.container {  
    display: grid;  
    grid-template-columns: repeat(3, 1fr);  
}
```

Meaning:

- 3 equal columns
- 1fr = equal space distribution

Example:

```
[1][2][3]  
[4][5][6]
```

5. Grid Rows

Rows define **vertical structure**

Syntax:

```
.container {  
    display: grid;  
    grid-template-rows: 100px 100px;  
}
```

Meaning:

- 2 rows
- Each row height = 100px

Row + Column Example

```
.container {  
    display: grid;  
    grid-template-columns: repeat(3, 1fr);  
    grid-template-rows: 100px 100px;  
}
```

Output:

[1][2][3]

[4][5][6]

6. Gap Property (Spacing)

Used to add space between grid items.

Syntax:

```
.container {  
    display: grid;  
    gap: 10px;  
}
```

Meaning:

- Space between rows and columns

7. Full Grid Example

```
<html>  
<head>  
<style>  
.container {  
    display: grid;  
    grid-template-columns: repeat(3, 1fr);  
    grid-template-rows: 100px 100px;  
    gap: 10px;  
}  
.box {  
    background: steelblue;  
    color: white;  
    display: flex;  
    justify-content: center;  
    align-items: center;  
    font-size: 20px;  
}  
</style>  
</head>
```

```
<body>
<div class="container">
  <div class="box">1</div>
  <div class="box">2</div>
  <div class="box">3</div>
  <div class="box">4</div>
  <div class="box">5</div>
  <div class="box">6</div>
</div>
</body>
</html>
```

8. Grid vs Flexbox

Feature	Flexbox	Grid
Direction	1D	2D
Rows + Columns	No	Yes
Best for	Navigation, alignment	Layout systems
Complexity	Easy	Advanced

9. Real Life Analogy

Grid Concept	Real Life
Grid container	Classroom
Columns	Student rows
Rows	Bench rows
Items	Students

Tasks (Day 10 Practice)

Task 1

Create grid container with 3 columns

Task 2

Create grid with 2 rows and 2 columns

Task 3

Use repeat(4, 1fr)

Task 4

Add gap between grid items

Task 5

Create simple product grid (6 boxes)

Task 6



Create image gallery using grid layout

Task 7

Design dashboard cards using grid

Task 8

Create uneven grid (different row sizes)

Task 9

Combine flexbox header + grid body layout

Task 10 (Challenge)

Create a mini eCommerce layout:

- Header (flexbox)
- Product grid (CSS Grid)
- Footer section

Mini Projects

Project 1: Product Grid Page

- 3-column product layout
- Cards with spacing

Project 2: Image Gallery

- Responsive grid images
- Equal spacing

Project 3: Admin Dashboard Layout

- Sidebar + grid content area
- Cards layout

Project 4: Website Home Layout

- Header (flex)
- Main content (grid)
- Footer (block)

Day 11

CSS Grid Advanced (Grid Area Layout)

1. What is Grid Area Layout?

Grid Area Layout is a **powerful feature of CSS Grid** that allows you to design a full webpage layout using named sections.

Instead of positioning elements manually, you can **name areas like:**

- header
- sidebar
- content
- footer

Simple Meaning:

Grid Area = "Draw your website layout like a map"

2. Why Grid Areas?

Without grid areas:

- You manually place items using row/column numbers

With grid areas:

- You design layout visually using names ✓

Very useful for:

- Dashboards
- Admin panels
- Full websites

3. Basic Structure of Grid Areas

You need 2 things:

1. Container (define layout map)

2. Items (assign names)

Step 1: Grid Container Layout Map

```
.container {  
  display: grid;  
  
  grid-template-areas:  
    "header header header"  
    "sidebar content content"  
    "footer footer footer";  
}
```

Meaning:



HEADER HEADER HEADER
SIDEBAR CONTENT CONTENT
FOOTER FOOTER FOOTER

Step 2: Assign Areas to Items

```
.header {  
  grid-area: header;  
}
```

```
.sidebar {  
  grid-area: sidebar;  
}
```

```
.content {  
  grid-area: content;  
}
```

```
.footer {  
  grid-area: footer;  
}
```

4. Full Example (Real Layout)

```
<html>  
<head>  
<style>
```

```
.container {  
  display: grid;  
  grid-template-areas:  
    "header header header"  
    "sidebar content content"  
    "footer footer footer";  
  
  grid-template-rows: 80px 1fr 60px;  
  grid-template-columns: 200px 1fr 1fr;  
  gap: 10px;  
  height: 100vh;  
}
```

```
.header {
```



```
grid-area: header;
background: steelblue;
color: white;
}
.sidebar {
  grid-area: sidebar;
  background: lightgray;
}
.content {
  grid-area: content;
  background: lightgreen;
}
.footer {
  grid-area: footer;
  background: black;
  color: white;
}
</style>
</head>
<body>
<div class="container">
  <div class="header">HEADER</div>
  <div class="sidebar">SIDEBAR</div>
  <div class="content">CONTENT AREA</div>
  <div class="footer">FOOTER</div>
</div>
</body>
</html>
```

5. How Grid Area Works

Area Name	Meaning
header	Top section
sidebar	Left menu
content	Main body
footer	Bottom section

6. Important Concepts

1. Repeating same area



"header header header"

☞ Means header spans full width

2. Empty space (optional)

"header header header"

"sidebar . content"

☞ . means empty space

3. Responsive control

You can change layout for mobile easily:

@media(max-width: 768px) {

.container {

grid-template-areas:

"header"

"content"

"sidebar"

"footer";

}

}

7. Real-Life Analogy

Grid Area	Real Life
header	Top banner
sidebar	Menu board
content	Main display
footer	Bottom notice

Tasks (Day 11 Practice)

Task 1

Create grid layout with header, sidebar, content, footer

Task 2

Change layout so header spans full width

Task 3

Create 2-column layout using grid areas

Task 4

Use . for empty grid space

Task 5

Style each grid area with different colors

Task 6

Create admin dashboard layout using grid areas



Task 7

Build website layout with fixed sidebar and content

Task 8

Convert flex layout to grid area layout

Task 9

Create responsive grid area layout (mobile version)

Task 10 (Challenge)

Build full website layout:

- Header
- Sidebar menu
- Main content section
- Footer

Must use grid-template-areas only

Mini Projects

Project 1: Website Layout Template

- Header
- Sidebar
- Content
- Footer
- Clean grid structure

Project 2: Admin Dashboard UI

- Sidebar navigation
- Main analytics section
- Footer

Project 3: E-commerce Layout

- Header
- Product grid section (inside content)
- Sidebar filters

Project 4: Portfolio Layout

- Header (name)
- Sidebar (skills/social links)
- Content (projects)
- Footer (contact)

Day 12: Responsive Design (Media Queries)

1. What is Responsive Design?

Responsive Design means:

Website should look good in all devices



- Mobile
- Tablet
- Laptop
- Desktop

Simple Meaning:

Same website → adjusts automatically for different screen sizes

Without Responsive Design:

- Text too big/small
- Layout broken
- Horizontal scrolling
- Bad user experience

With Responsive Design:

- Perfect alignment
- Adjusted fonts
- Flexible layout
- Mobile-friendly UI

3. What are Media Queries?

Media Queries are CSS rules that apply styles based on:

Screen size

Device width

Resolution

Syntax:

```
@media (condition) {  
  /* CSS styles */  
}
```

3. Basic Media Query Example

```
body {  
  background-color: white;  
}
```

```
@media (max-width: 768px) {  
  body {  
    background-color: lightgray;  
  }  
}
```

Meaning:

- Default → white background

- Screen $\leq 768\text{px}$ $\rightarrow$ gray background

4. Breakpoints (VERY IMPORTANT)

Breakpoints = screen size limits

Common Breakpoints:

Device	Size
Mobile	0 - 576px
Tablet	576px - 768px
Laptop	768px - 1024px
Desktop	1024px+

5. Responsive Text Example

```
h1 {  
  font-size: 40px;  
}  
@media (max-width: 768px) {  
  h1 {  
    font-size: 25px;  
  }  
}
```

Meaning:

- Big text in desktop
- Smaller text in mobile

6. Responsive Layout Example (Flexbox)

```
.container {  
  display: flex;  
}  
  
@media (max-width: 768px) {  
  .container {  
    flex-direction: column;  
  }  
}
```

Meaning:

- Desktop $\rightarrow$ row layout
- Mobile $\rightarrow$ column layout

7. Responsive Grid Example

```
.container {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
}
```

```
@media (max-width: 768px) {
  .container {
    grid-template-columns: 1fr;
  }
}
```

Meaning:

- Desktop → 3 columns
- Mobile → 1 column

8. Mobile First Approach (IMPORTANT ★)

Design for mobile first, then scale up

Example:

```
.box {
  width: 100%;
}
/* Tablet */
@media (min-width: 768px) {
  .box {
    width: 50%;
  }
}
/* Desktop */
@media (min-width: 1024px) {
  .box {
    width: 25%;
  }
}
```

9. Real Life Analogy

Device	Layout Behavior
Mobile	Single column
Tablet	2 columns
Desktop	Multi columns

10. Full Responsive Example

```
<html>
<head>
<style>
.container {
  display: flex;
  gap: 10px;
}
.box {
  flex: 1;
  background: steelblue;
  color: white;
  padding: 20px;
  text-align: center;
}
/* Mobile View */
@media (max-width: 768px) {
  .container {
    flex-direction: column;
  }
}
</style>
</head>

<body>
<div class="container">
  <div class="box">Box 1</div>
  <div class="box">Box 2</div>
  <div class="box">Box 3</div>
</div>
</body>
</html>
```

Tasks (Day 12 Practice)

Task 1

Change background color using media query

Task 2

Make heading smaller on mobile



Task 3

Convert flex row → column in mobile view

Task 4

Create responsive 2-column layout

Task 5

Make image width responsive (100%)

Task 6

Create responsive navbar (desktop row, mobile column)

Task 7

Build responsive card layout

Task 8

Create responsive grid (3 columns → 1 column)

Task 9

Design mobile-first layout

Task 10 (Challenge)

Create full responsive webpage:

- Header
- Cards section
- Footer

Must work in mobile + desktop

Mini Projects

Project 1: Responsive Profile Page

- Mobile friendly layout
- Image + text stacking

Project 2: Responsive Product Cards

- Desktop: 3 columns
- Mobile: 1 column

Project 3: Responsive Landing Page

- Header + content + footer
- Fully responsive

Project 4: Admin Dashboard Responsive UI

- Sidebar collapses in mobile
- Content adjusts automatically



Day 13

Mini Project – Static Web Page (Full Website Build)

Project Title: Simple Company Website (Static Web Page)

This project combines everything you learned:

- HTML structure
- Semantic tags
- CSS styling
- Flexbox
- Grid (basic usage)
- Responsive design

Project Goal

Build a **professional static website** with:

- Header (logo + title)
- Navigation bar
- Hero section
- About section
- Services section
- Team section
- Contact section
- Footer

1. Project Structure

static-website/

```
|
|— index.html
|— style.css
```

2. HTML Code (index.html)

```
<!DOCTYPE html>
<html>
<head>
  <title>Company Website</title>
  <link rel="stylesheet" href="style.css">
</head>
<body>

<header>
```



```
<h1>Tech Solutions</h1>
<p>We Build Modern Web Applications</p>
</header>

<nav>
  <a href="#home">Home</a>
  <a href="#about">About</a>
  <a href="#services">Services</a>
  <a href="#team">Team</a>
  <a href="#contact">Contact</a>
</nav>

<section id="home" class="hero">
  <h2>Welcome to Tech Solutions</h2>
  <p>Your success starts with a website</p>
  <button>Get Started</button>
</section>
<section id="about">
  <h2>About Us</h2>
  <p>We are a professional web development company providing modern and
responsive websites.</p>
</section>
<section id="services">
  <h2>Our Services</h2>
  <div class="cards">
    <div class="card">
      <h3>Web Development</h3>
      <p>We build responsive websites</p>
    </div>
    <div class="card">
      <h3>UI/UX Design</h3>
      <p>Modern and clean UI designs</p>
    </div>
    <div class="card">
      <h3>SEO Optimization</h3>
      <p>Improve your website ranking</p>
    </div>
  </div>
</section>
```

```
</div>
</section>
<section id="team">
  <h2>Our Team</h2>
  <div class="cards">
    <div class="card">Developer 1</div>
    <div class="card">Developer 2</div>
    <div class="card">Designer</div>
  </div>
</section>
<section id="contact">
  <h2>Contact Us</h2>
  <p>Email: contact@techsolutions.com</p>
  <p>Phone: +91 98765 43210</p>
</section>
<footer>
  <p>© 2026 Tech Solutions. All Rights Reserved.</p>
</footer>
</body>
</html>
```

3. CSS Code (style.css)

```
body {
  margin: 0;
  font-family: Arial;
}
/* HEADER */
header {
  background: steelblue;
  color: white;
  text-align: center;
  padding: 20px;
}
/* NAVBAR */
nav {
  display: flex;
  justify-content: center;
```



```
background: black;
}
nav a {
  color: white;
  padding: 15px;
  text-decoration: none;
}
nav a:hover {
  background: gray;
}
/* HERO SECTION */
.hero {
  text-align: center;
  padding: 60px;
  background: lightblue;
}
.hero button {
  padding: 10px 20px;
  border: none;
  background: black;
  color: white;
  cursor: pointer;
}
/* SECTIONS */
section {
  padding: 40px;
}
/* CARDS */
.cards {
  display: flex;
  gap: 20px;
}
.card {
  flex: 1;
  padding: 20px;
  background: lightgray;
  text-align: center;
```



```
border-radius: 5px;
}
/* FOOTER */
footer {
  text-align: center;
  background: black;
  color: white;
  padding: 10px;
}
/* RESPONSIVE */
@media (max-width: 768px) {
  nav {
    flex-direction: column;
  }
  .cards {
    flex-direction: column;
  }
  .hero {
    padding: 30px;
  }
}
```

Features You Learned in This Project

- Semantic HTML structure
- Navigation bar using Flexbox
- Card layout system
- Section-based website design
- Button styling
- Responsive design using media queries

Output

Desktop:

- Horizontal navbar
- Cards in row (3 columns)
- Full hero section

Mobile:

- Navbar becomes vertical
- Cards stack vertically
- Responsive padding



Tasks (Day 13)

Task 1

Change website name and branding

Task 2

Add image inside hero section

Task 3

Change navbar colors and hover effects

Task 4

Add icons in services section

Task 5

Increase spacing between sections

Task 6

Convert cards to grid layout instead of flexbox

Task 7

Add contact form instead of text

Task 8

Make hero section full screen height

Task 9

Add shadow effect to cards

Task 10 (Challenge)

Convert this website into:

Dark theme + smooth hover animations

Mini Improvements

1: Add animations (hover effects)

2: Add sticky navbar

3: Add scroll smooth effect

4: Add image gallery section

Day 14

Revision + Practice (HTML + CSS Full Review Day)

1. What You Learned Till Now

This day is for **strong revision + coding practice**, not new topics.

HTML REVISION

Basic Structure

- `<!DOCTYPE html>`
- `<html>`, `<head>`, `<body>`

Important Tags



- Headings → <h1> to <h6>
- Paragraph → <p>
- Link → <a>
- Image →
- List → , ,
- Table → <table>, <tr>, <th>, <td>

Forms

- <input> types (text, email, password, number)
- <textarea>
- <radio>, <checkbox>
- required, min, max

Semantic Tags

- <header>
- <nav>
- <section>
- <article>
- <footer>

CSS REVISION

CSS Types

- Inline CSS
- Internal CSS
- External CSS

Selectors

- Element → h1 {}
- Class → .box {}
- ID → #title {}

Box Model

- Content
- Padding
- Border
- Margin

Layout

- display: block / inline / inline-block / none
- Position:
 - relative
 - absolute
 - fixed



- sticky

Flexbox

- display: flex
- justify-content
- align-items
- gap

Grid

- display: grid
- grid-template-columns
- grid-template-rows
- grid-template-areas

Responsive Design

- Media Queries

@media (max-width: 768px) { }

2. PRACTICE SET

1: HTML Basics

Create a webpage with:

- Name (h1)
- About paragraph
- Image
- Link to Google

Practice 2: Form

Create a registration form:

- Name
- Email
- Password
- Gender
- Submit button

Practice 3: Table

Create student marks table:

- Name
- Subject
- Marks

Practice 4: CSS Styling

Style a page with:

- Background color
- Font size change



- Text color
- Button styling

Practice 5: Flexbox

Create navbar:

- Logo left
- Menu center/right
- Use justify-content

Practice 6: Grid

Create product grid:

- 3 columns desktop
- 1 column mobile

Practice 7: Responsive Design

Make any page mobile-friendly using media query.

Practice 8

Build from scratch:

Mini Website

Include:

- Header
- Navbar
- Hero section
- About section
- Services (cards)
- Contact section
- Footer

Must use:

- Flexbox
- Grid
- Media Queries
- Semantic tags

Day 15

JavaScript Introduction + Variables

1. What is JavaScript?

JavaScript is a **programming language** used to make websites interactive.

HTML = Structure

CSS = Design

JavaScript = Logic + Actions